Syllabus

II Year - IV Semester: B. Tech. Computer Science and Engineering (Cyber Security)

244CY4-24: Web Application Security Lab

Credit: 1 **Max. Marks: 100 (IA: 60, ETE: 40)**

0L+0T+2P **End Term Exam: 2 Hours**

S.NO	CONTENTS
1	Understanding the History of Software Security, Recognizing Web Application Security Threats.
2	Perform vulnerability testing on a web application.
3	Understand and secure session-based authentication using cookies, identify common threats to REST APIs and apply security controls.
4	Understand and implement each phase of the Vulnerability Assessment Lifecycle.
5	Perform a security assessment of an Android app..
6	Understand the impact and mechanics of social engineering attacks.
7	Demonstrate SQL Injection and how to prevent it
8	Identify flaws in session and authentication handling.
9	Use Comodo scan for web vulnerabilities.
10	Use Burp Suite for intercepting and testing web traffic.

Experiment-1

AIM: Understanding the History of Software Security, Recognizing Web Application Security Threats.

The history of Software Security-Recognizing Web Application Security Threats The history of software security, particularly in the context of recognizing web application security threats, is marked by the evolution of technology and the increasing sophistication of cyber threats. Below is a brief overview of key milestones and developments in the history of web application security:

1. **1990s - The Emergence of the World Wide Web:**- The World Wide Web became publicly accessible, leading to the proliferation of websites and web applications.- The focus during this period was on building and expanding the web, with limited attention to security considerations.

2. **Late 1990s to Early 2000s - Rise of Common Vulnerabilities:**- As web applications grew in complexity, security vulnerabilities became more apparent.- Common vulnerabilities such as buffer overflows, SQL injection, and cross-site scripting (XSS) started to emerge.- The concept of input validation gained attention as a crucial aspect of web application security.

3. **2002 - The Birth of OWASP:** - The Open Web Application Security Project (OWASP) was founded to provide resources and guidelines for improving web application security.- The OWASP Top Ten, a list of the most critical web application security risks, was introduced to raise awareness about common vulnerabilities.

4. **Mid-2000s - Proliferation of Web 2.0 and AJAX:**- The advent of Web 2.0 technologies and the use of Asynchronous JavaScript and XML (AJAX) introduced new attack vectors.- Security researchers and attackers began to exploit vulnerabilities related to the dynamic and interactive nature of these technologies.

5. **Late 2000s - Increased Focus on Secure Development Practices:**- Organizations started recognizing the importance of integrating security into the software development life cycle.- Secure coding practices and tools became more prevalent, emphasizing the need to consider security from the design phase onward.

6. **2010s - Evolution of Threat Landscape:**- The threat landscape continued to evolve with the rise of mobile applications, cloud computing, and APIs.- Web security standards like HTTP Strict Transport Security (HSTS) and Content Security Policy (CSP) gained adoption to enhance protection against various

- Introduction of security standards like:
    - OWASP
    - CERT
- Common web threats:
    - SQL Injection
    - Cross-Site Scripting (XSS)
    - Cross-Site Request Forgery (CSRF)
    - Broken Authentication
    - Security Misconfiguration

Tools

- Browser
- OWASP Top 10 Documentation

Experiment-1(a)

Aim:

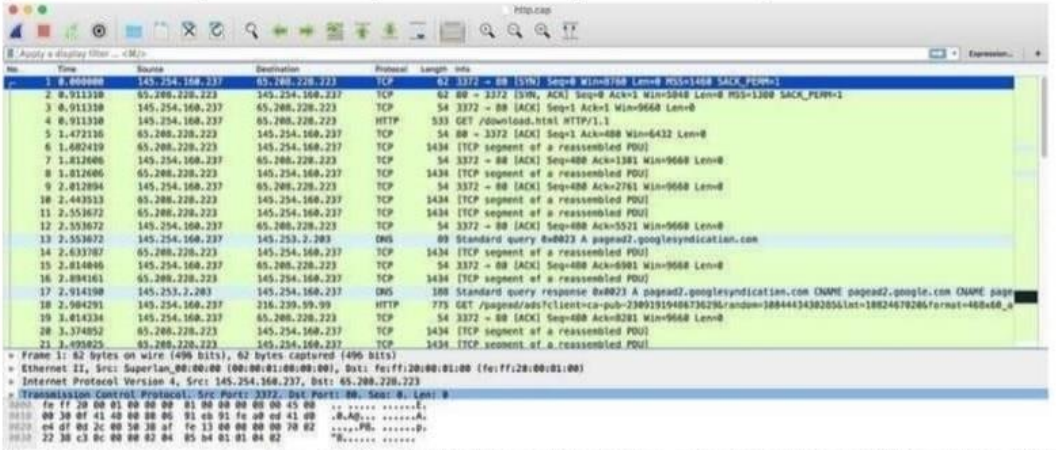
To explore the Analyze the difference between HTTP vs HTTPS by using wireshark

Installation of Wireshark Software

- Open the web browser.
- Search for 'Download Wireshark.'
- Select the Windows installer according to your system configuration, either 32-bt or 64-bit. Save the program and close the browser.
- Now, open the software, and follow the install instruction by accepting the license.
- The Wireshark is ready for use.

HTTP capture

Before start analyzing any packet, please turn off "Allow subdissector to reassemble TCP streams"(Preference → Protocol → TCP)(This will prevent TCP packet to split into multiple PDU unit)



As you can see I am using HTTP so that the encryption will not be hidden behind TLS.

As you can see at line number 13 standard DNS resolution is happening.

```
13 2.553672   145.254.160.237   145.253.2.203     DNS   89  Standard query 0x0023 A pagead2.googlesyndication.com
14 2.633787   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
15 2.814046   145.254.160.237   65.208.228.223    TCP   54  3372 → 80 [ACK] Seq=400 Ack=6901 Win=9660 Len=0
16 2.894161   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
17 2.914190   145.253.2.203     145.254.160.237   DNS   188 Standard query response 0x0023 A pagead2.googlesyndication.com CNAME pagead2.google.com CNAME page
18 2.984291   145.254.160.237   216.239.59.99     HTTP  775 GET /pagead/ads?client=ca-pub-2309191948673629&random=1084443430285&lmt=1082467020&format=468x60_a
19 3.014334   145.254.160.237   65.208.228.223    TCP   54  3372 → 80 [ACK] Seq=400 Ack=8281 Win=9660 Len=0
20 3.374852   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
21 3.495025   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
▸ User Datagram Protocol, Src Port: 3009, Dst Port: 53
▾ Domain Name System (query)
    [Response In: 17]
    Transaction ID: 0x0023
  ▸ Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
  ▸ Queries
```

In line number 17 you see the response we are getting back with full DNS resolution

```
17 2.914190   145.253.2.203     145.254.160.237   DNS   188 Standard query response 0x0023 A pagead2.googlesyndication.com CNAME pagead2.google.com CNAME page
18 2.984291   145.254.160.237   216.239.59.99     HTTP  775 GET /pagead/ads?client=ca-pub-2309191948673629&random=1084443430285&lmt=1082467020&format=468x60_a
19 3.014334   145.254.160.237   65.208.228.223    TCP   54  3372 → 80 [ACK] Seq=400 Ack=8281 Win=9660 Len=0
20 3.374852   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
21 3.495025   65.208.228.223    145.254.160.237   TCP   1434 [TCP segment of a reassembled PDU]
    Transaction ID: 0x0023
  ▸ Flags: 0x8180 Standard query response, No error
    Questions: 1
    Answer RRs: 4
    Authority RRs: 0
    Additional RRs: 0
  ▾ Queries
    ▸ pagead2.googlesyndication.com: type A, class IN
  ▾ Answers
    ▸ pagead2.googlesyndication.com: type CNAME, class IN, cname pagead2.google.com
    ▸ pagead2.google.com: type CNAME, class IN, cname pagead.google.akadns.net
    ▸ pagead.google.akadns.net: type A, class IN, addr 216.239.59.104
    ▸ pagead.google.akadns.net: type A, class IN, addr 216.239.59.99
```

Now if you look at Packet number 4 i.e is get request,HTTP primarily used two command

1: GET: To retrieve information

2: POST: To send information(For eg: when we submit some form we fill some data i.e is POST)

```
4 0.911310    145.254.160.237   65.208.228.223    HTTP  533 GET /download.html HTTP/1.1
5 1.472116    65.208.228.223    145.254.160.237   TCP   54  80 → 3372 [ACK] Seq=1 Ack=480 Win=6432 Len=0
6 1.682419    65.208.228.223    145.254.160.237   HTTP/… 1434 HTTP/1.1 200 OK
7 1.812686    145.254.160.237   65.208.228.223    TCP   54  3372 → 80 [ACK] Seq=400 Ack=1381 Win=9660 Len=0
8 1.812686    65.208.228.223    145.254.160.237   HTTP  1434 Continuation
9 2.012894    145.254.160.237   65.208.228.223    TCP   54  3372 → 80 [ACK] Seq=400 Ack=2761 Win=9660 Len=0
10 2.443513   65.208.228.223    145.254.160.237   HTTP  1434 Continuation
▸ Frame 4: 533 bytes on wire (4264 bits), 533 bytes captured (4264 bits)
▸ Ethernet II, Src: Superlan_00:00:00 (00:00:01:00:00:00), Dst: fe:ff:20:00:01:00 (fe:ff:20:00:01:00)
▸ Internet Protocol Version 4, Src: 145.254.160.237, Dst: 65.208.228.223
▸ Transmission Control Protocol, Src Port: 3372, Dst Port: 80, Seq: 1, Ack: 1, Len: 479
▾ Hypertext Transfer Protocol
  ▸ GET /download.html HTTP/1.1\r\n
    Host: www.ethereal.com\r\n
    User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.6) Gecko/20040113\r\n
    Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1\r\n
    Accept-Language: en-us,en;q=0.5\r\n
    Accept-Encoding: gzip,deflate\r\n
    Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
    Keep-Alive: 300\r\n
    Connection: keep-alive\r\n
    Referer: http://www.ethereal.com/development.html\r\n
    \r\n
    [Full request URI: http://www.ethereal.com/download.html]
    [HTTP request 1/1]
    [Response in frame: 6]
```

Here I am trying to get download.html via HTTP protocol 1.1(The new version of protocol is now available i.e 2.0)

Then at line number 5 we see the acknowledgment as well as line number 6 server was able to found that page and send HTTP status code 200.

If you want more info about HTTP status code

HTTP Status Codes

You will see some more info like for packet 6, like Server type is Apache, content type is HTML, how long is the content length is,

```
6 1.682419   65.208.228.223   145.254.160.237   HTTP/…   1434 HTTP/1.1 200 OK
7 1.812606   145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=1381 Win=9660 Len=0
8 1.812606   65.208.228.223   145.254.160.237   HTTP     1434 Continuation
9 2.012894   145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=2761 Win=9660 Len=0
10 2.443513  65.208.228.223   145.254.160.237   HTTP     1434 Continuation

▸ Frame 6: 1434 bytes on wire (11472 bits), 1434 bytes captured (11472 bits)
▸ Ethernet II, Src: fe:ff:20:00:01:00 (fe:ff:20:00:01:00), Dst: Superlan_00:00:00 (00:00:01:00:00:00)
▸ Internet Protocol Version 4, Src: 65.208.228.223, Dst: 145.254.160.237
▸ Transmission Control Protocol, Src Port: 80, Dst Port: 3372, Seq: 1, Ack: 480, Len: 1380
▾ Hypertext Transfer Protocol
  ▸ HTTP/1.1 200 OK\r\n
    Date: Thu, 13 May 2004 18:17:12 GMT\r\n
    Server: Apache\r\n
    Last-Modified: Tue, 20 Apr 2004 13:17:00 GMT\r\n
    ETag: "9a01a-4096-7e354b00"\r\n
    Accept-Ranges: bytes\r\n
  ▸ Content-Length: 18070\r\n
    Keep-Alive: timeout=15, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: text/html; charset=ISO-8859-1\r\n
    \r\n
    [HTTP response 1/1]
    [Time since request: 0.771109000 seconds]
    [Request in frame: 4]
    File Data: 1086 bytes
```

Then you will see bunch of continuation that is due to TCP window where you don't get acknowledgement for each and every packet

```
6 1.682419   65.208.228.223   145.254.160.237   HTTP/…   1434 HTTP/1.1 200 OK
7 1.812606   145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=1381 Win=9660 Len=0
8 1.812606   65.208.228.223   145.254.160.237   HTTP     1434 Continuation
9 2.012894   145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=2761 Win=9660 Len=0
10 2.443513  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
11 2.553672  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
12 2.553672  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=5521 Win=9660 Len=0
13 2.553672  145.254.160.237   145.253.2.203    DNS        89 Standard query 0x0023 A pagead2.googlesyndication.com
14 2.633787  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
15 2.814046  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=6901 Win=9660 Len=0
16 2.894161  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
17 2.914190  145.253.2.203    145.254.160.237   DNS       188 Standard query response 0x0023 A pagead2.googlesyndication.com CNAME pagead2.google.com CNAME page
18 2.984291  145.254.160.237   216.239.59.99    HTTP      775 GET /pagead/ads?client=ca-pub-2309191948673629&random=1084443430205&lmt=1082467026&format=468x60_a
19 3.014334  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=8281 Win=9660 Len=0
20 3.374852  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
21 3.495025  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
22 3.495025  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=11041 Win=9660 Len=0
23 3.635227  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
24 3.645241  216.239.59.99    145.254.160.237   TCP        54 80 → 3371 [ACK] Seq=1 Ack=722 Win=31460 Len=0
25 3.815486  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=12421 Win=9660 Len=0
26 3.915630  216.239.59.99    145.254.160.237   HTTP     1484 HTTP/1.1 200 OK  (text/html)
27 3.955688  216.239.59.99    145.254.160.237   HTTP      214 Continuation
28 3.955688  145.254.160.237   216.239.59.99    TCP        54 3371 → 80 [ACK] Seq=722 Ack=1591 Win=8760 Len=0
29 4.185904  65.208.228.223   145.254.160.237   HTTP     1434 Continuation
30 4.216062  145.254.160.237   65.208.228.223   TCP        54 3372 → 80 [ACK] Seq=480 Ack=13081 Win=9660 Len=0
```

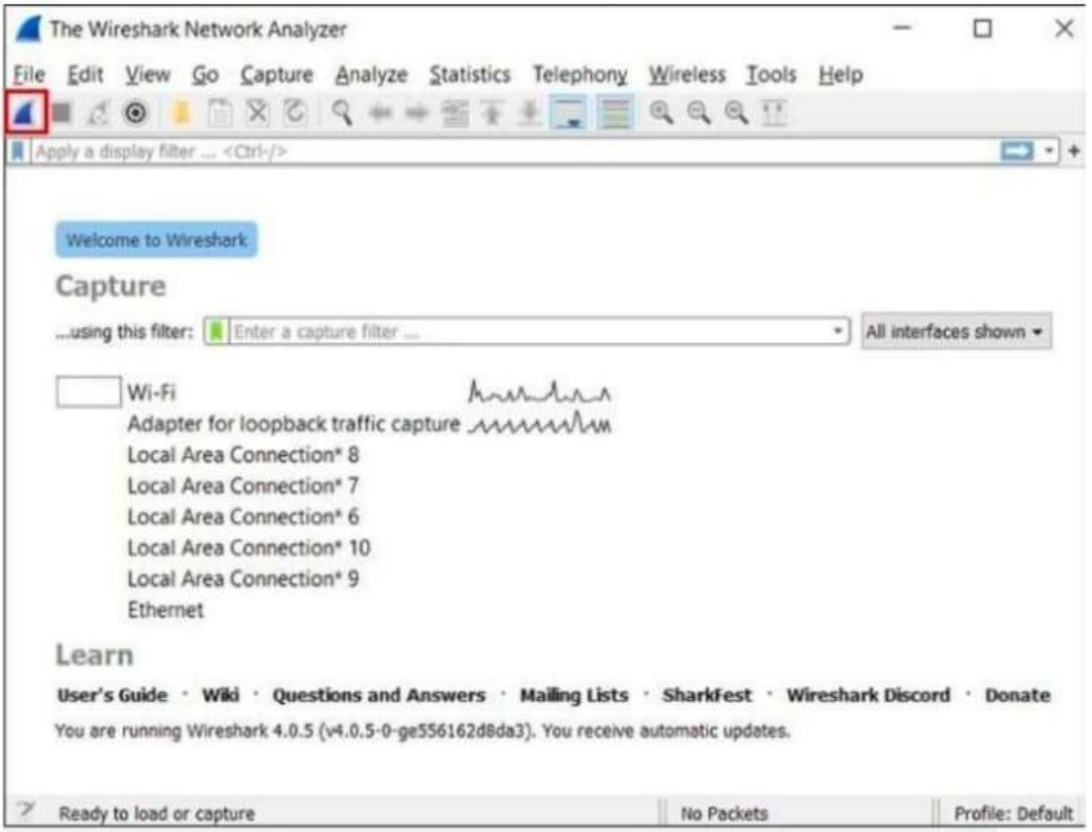
and at that top some usual TCP handshake

```
1 0.000000   145.254.160.237   65.208.228.223   TCP   62 3372 → 80 [SYN] Seq=0 Win=8760 Len=0 MSS=1460 SACK_PERM=1
2 0.911310   65.208.228.223   145.254.160.237   TCP   62 80 → 3372 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1380 SACK_PERM=1
3 0.911310   145.254.160.237   65.208.228.223   TCP   54 3372 → 80 [ACK] Seq=1 Ack=1 Win=9660 Len=0
```

capture HTTPS in Wireshark

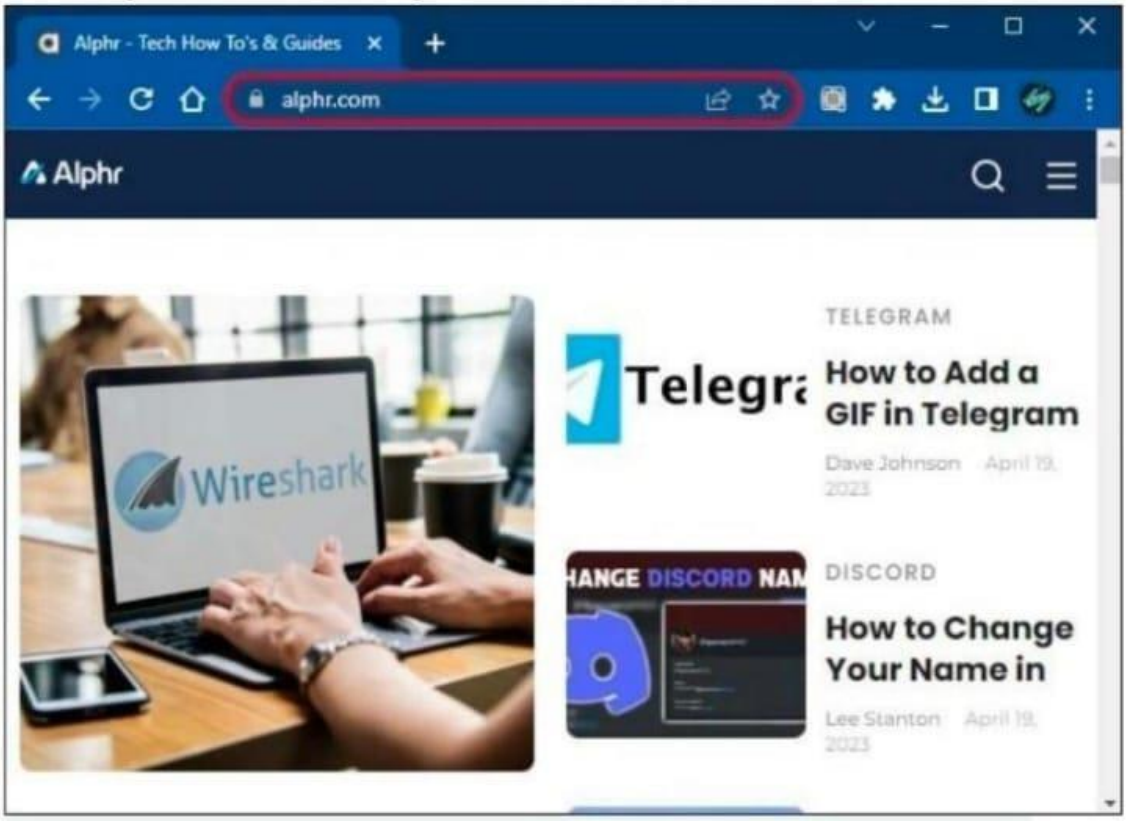
Capture and Decrypt Session Keys

1. Launch Wireshark and start an unfiltered capture session.

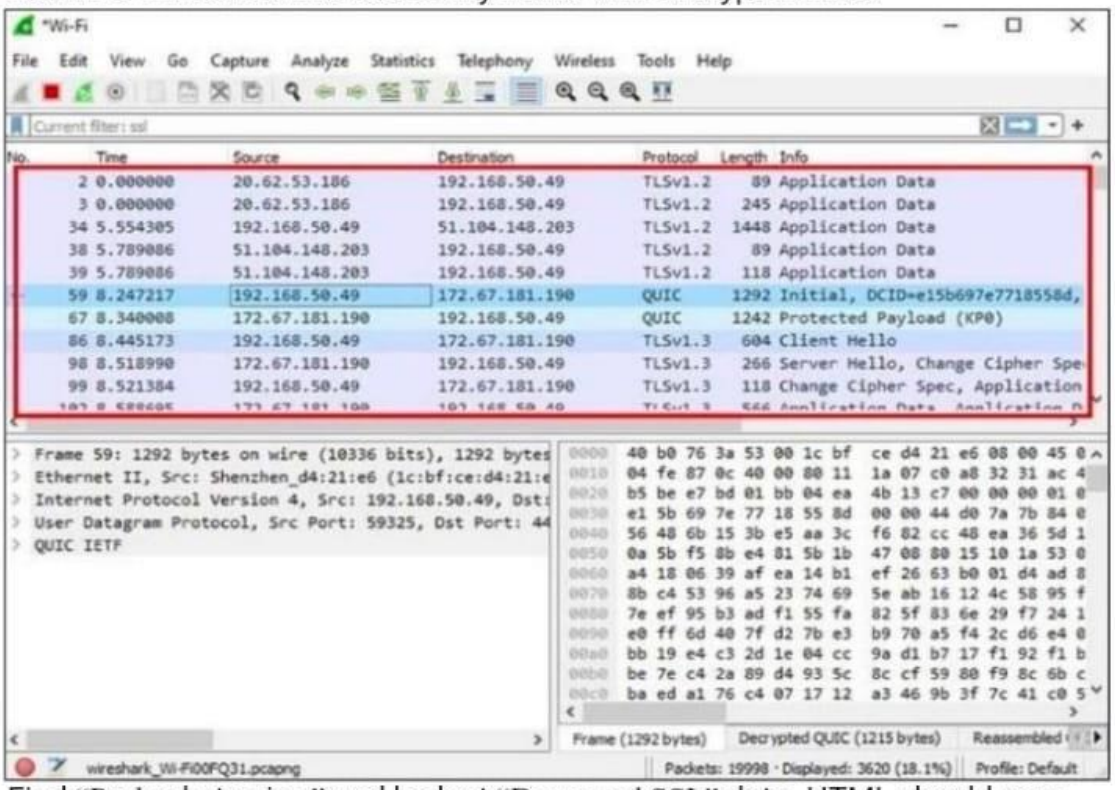


2. Minimize the Wireshark window and open your browser.

3. Go to any secure website to get data.



4. Return to Wireshark and select any frame with encrypted data.



5. Find "Packet byte view" and look at "Decrypted SSL" data. HTML should now be visible.

Conclusion

HTTPS provides better security than HTTP due to the following reasons:

- Encryption: The use of SSL/TLS encryption in HTTPS ensures that the data being transmitted between the client and the server is secure and cannot be intercepted by an attacker.
- Authentication: HTTPS uses digital certificates to authenticate the identity of the website, making it harder for attackers to impersonate a website and carry out a man-in-the-middle attack.
- Integrity: HTTPS provides data integrity which ensures that the data being transmitted between the client and the server has not been modified or tampered with.
- SEO: HTTPS is now a ranking signal for search engines, and having an HTTPS site can improve your search engine ranking.

Result

Experiment-1(b)

Aim:

To Analyze the various security mechanisms embedded with different protocols by using wireshark

Analyzing the security mechanisms embedded in different protocols using Wireshark involves examining network traffic captures to understand how various security features are implemented and whether they are effective. Below are some common security mechanisms embedded in different protocols that you can analyze using Wireshark:

1. Transport Layer Security (TLS/SSL):

- TLS/SSL is used to encrypt and secure data transmission over the network.
- In Wireshark, you can identify TLS/SSL traffic by looking at the protocol field, which should indicate TLS or SSL.
- You can analyze the TLS handshake process, certificate exchange, and the encryption ciphers being used.

2. Secure Shell (SSH):

- SSH is a secure protocol for remote access and data exchange.
- Wireshark can capture SSH traffic, and you can analyze the key exchange, authentication methods, and encryption algorithms being used.

3. IPsec (Internet Protocol Security):

- IPsec is used to secure IP communication through encryption and authentication.
- Wireshark can capture and analyze IPsec packets to understand the negotiation of security associations (SAs), encryption algorithms, and authentication methods.

4. HTTP Secure (HTTPS):

- HTTPS is a secure version of HTTP that uses TLS/SSL for encryption.
- Wireshark can capture HTTPS traffic and allow you to examine the encrypted content after decryption by examining the TLS layer.

5. Virtual Private Network (VPN) Protocols:

- VPN protocols like OpenVPN, L2TP, or PPTP provide secure tunnels for data transmission.
- Wireshark can capture VPN traffic, and you can analyze the encapsulation, authentication, and encryption mechanisms used within the VPN tunnel.

6. Secure Sockets Layer (SSL) VPN:

- SSL VPNs create secure connections for remote access.
- Wireshark can capture SSL VPN traffic and allow you to analyze the SSL handshake, certificate exchange, and data transmission.

7. Kerberos:

- Kerberos is a network authentication protocol.
- You can capture and analyze Kerberos traffic in Wireshark to observe the ticket granting process, authentication, and encryption.

8. SNMPv3 (Simple Network Management Protocol version 3):

- SNMPv3 includes security features such as authentication and encryption.
- Wireshark can help you understand how SNMPv3 is used to secure network management.

9. DNSSEC (Domain Name System Security Extensions):

- DNSSEC adds security to the DNS by signing DNS records.
- Wireshark can be used to capture DNSSEC-signed DNS queries and responses to validate the digital signatures.

10. OAuth and OAuth2:

- OAuth is used for delegated authorization, often in web and API interactions.
- You can capture OAuth flows and analyze the token exchange and authentication mechanisms.

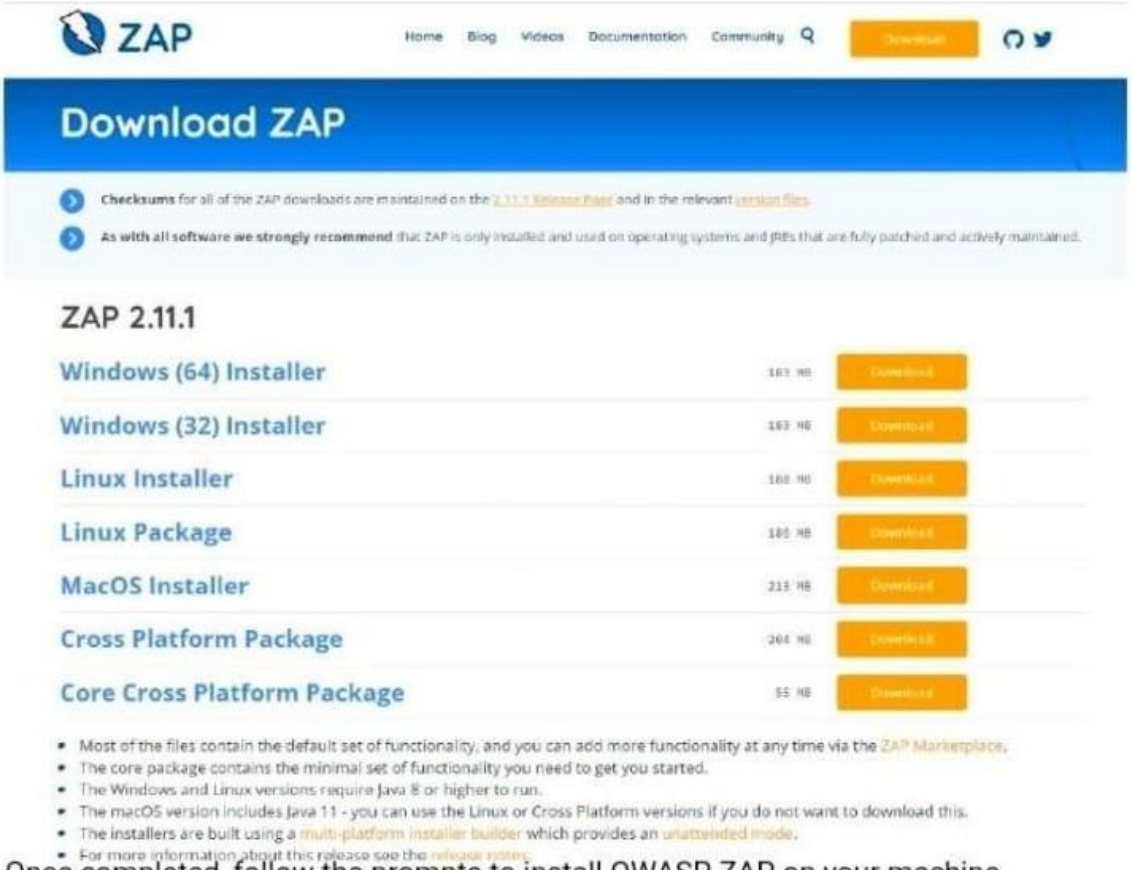
When analyzing security mechanisms using Wireshark, it's essential to focus on packet captures related to the specific protocol or technology you're interested in. Look for indicators of encryption, authentication, key exchanges, and other security features to assess the effectiveness and security of the implementation. Additionally, pay attention to any anomalies or potential vulnerabilities that may be present in the captured traffic.

Experiment -2

Aim: Perform vulnerability testing on a web application.

1. Installing ZAP

You can download the latest version from the OWASP ZAP website for your operating system to install ZAP or reference the ZAP docs for a more detailed installation guide.

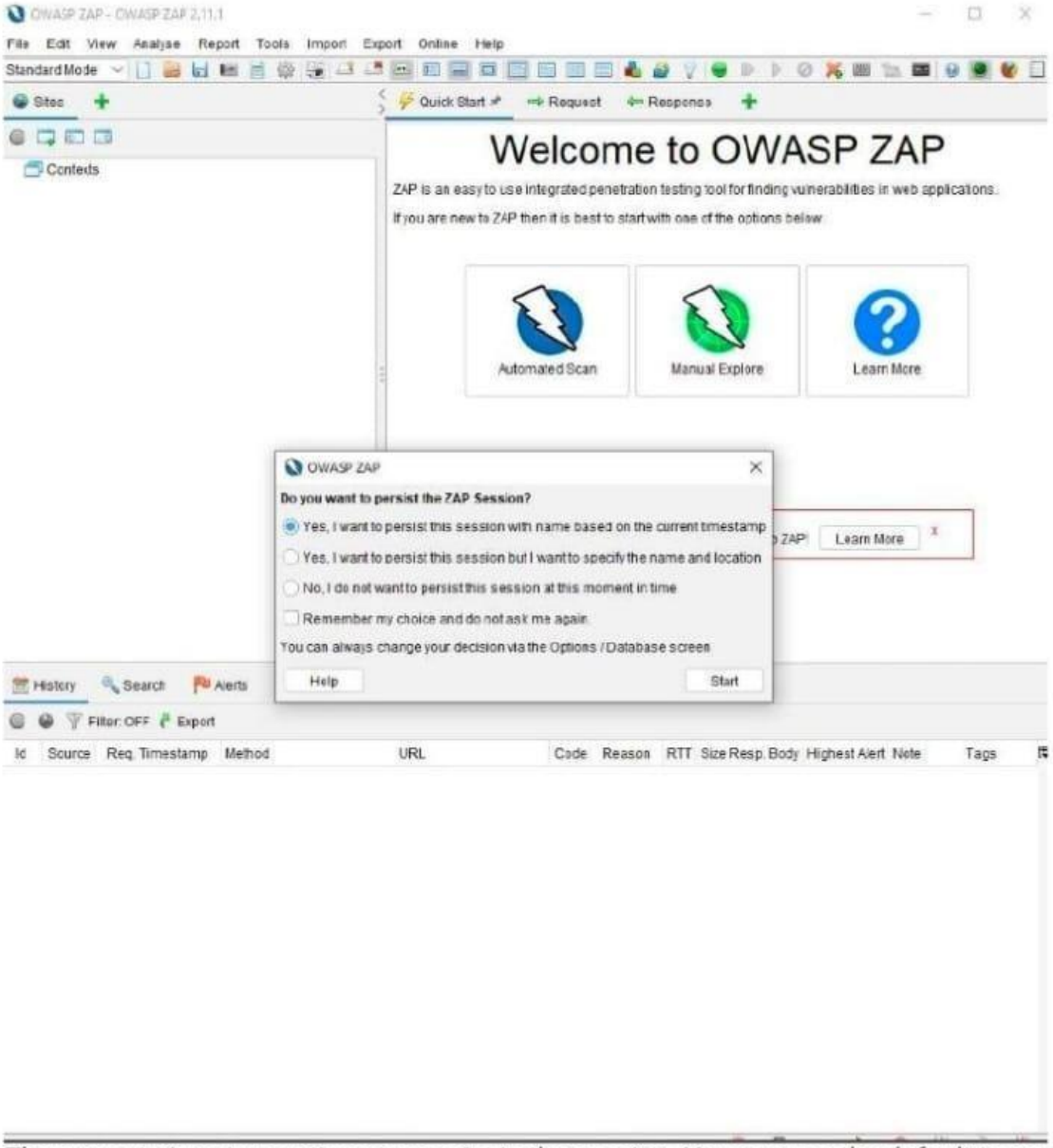


Once completed, follow the prompts to install OWASP ZAP on your machine.

2. Persisting a session

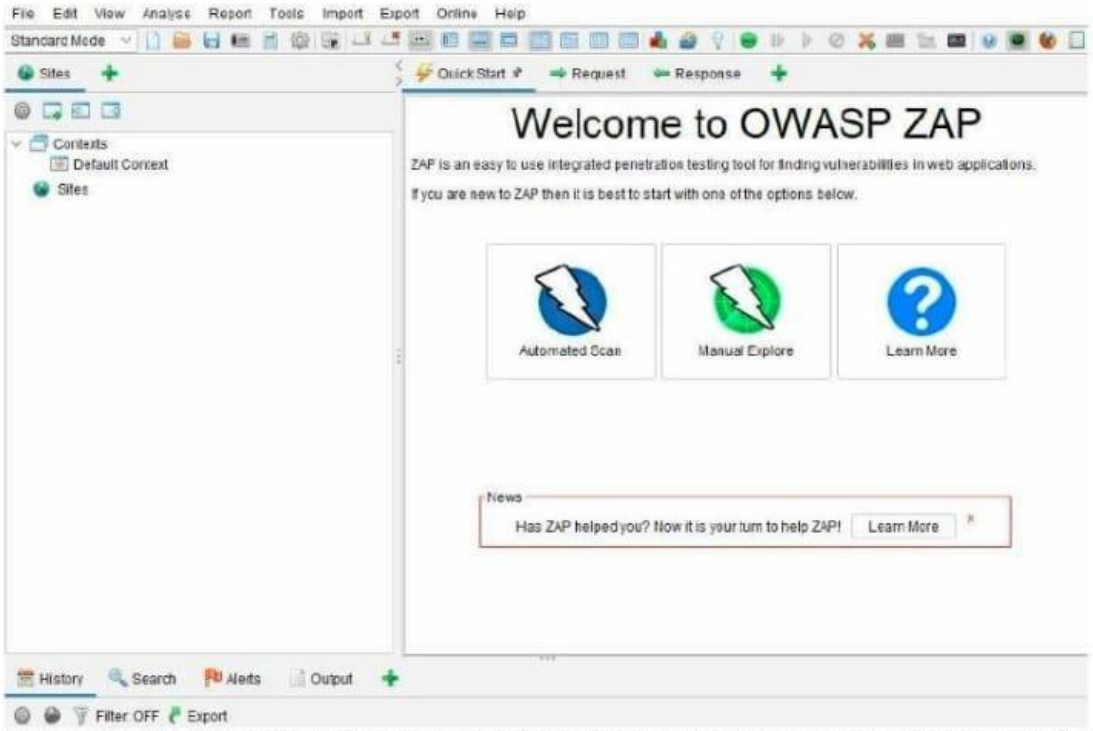
Persisting a session in OWASP ZAP means that the session will be saved and can be reopened at a later time. This is useful if you want to continue testing a website or application at a later time.

Once you've started OWASP ZAP, you will see a screen that looks like this:

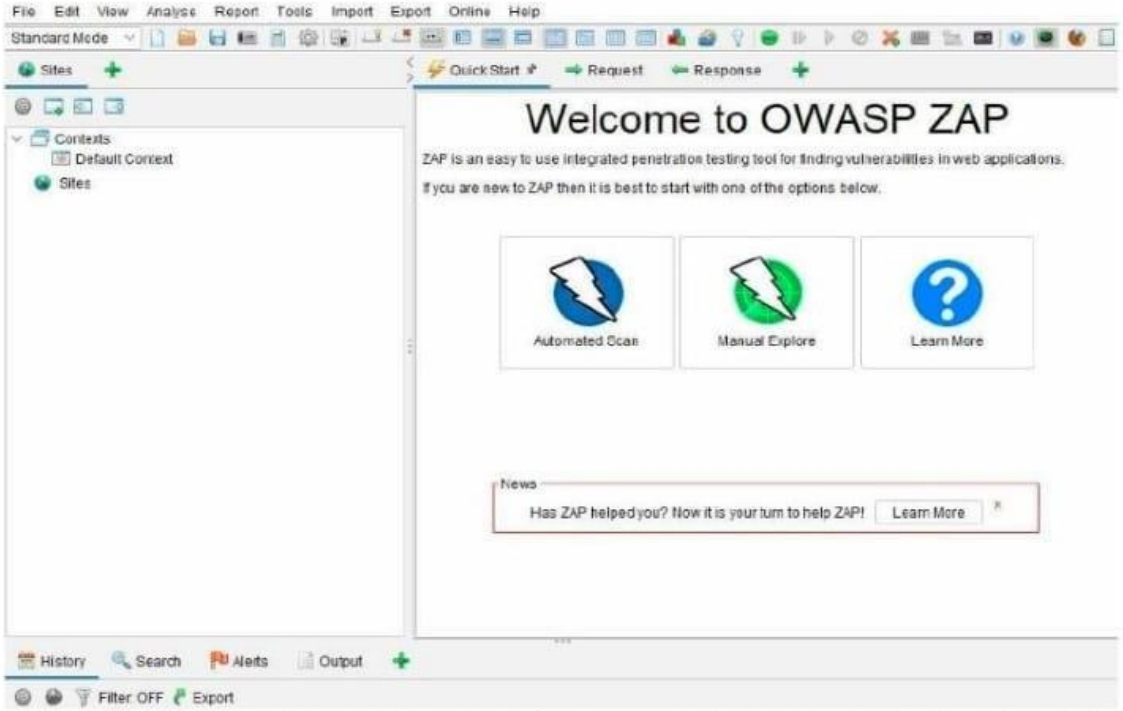


The prompt gives two options to persist in the session. You can use the default to name the session based on the current timestamp or set your name and location.

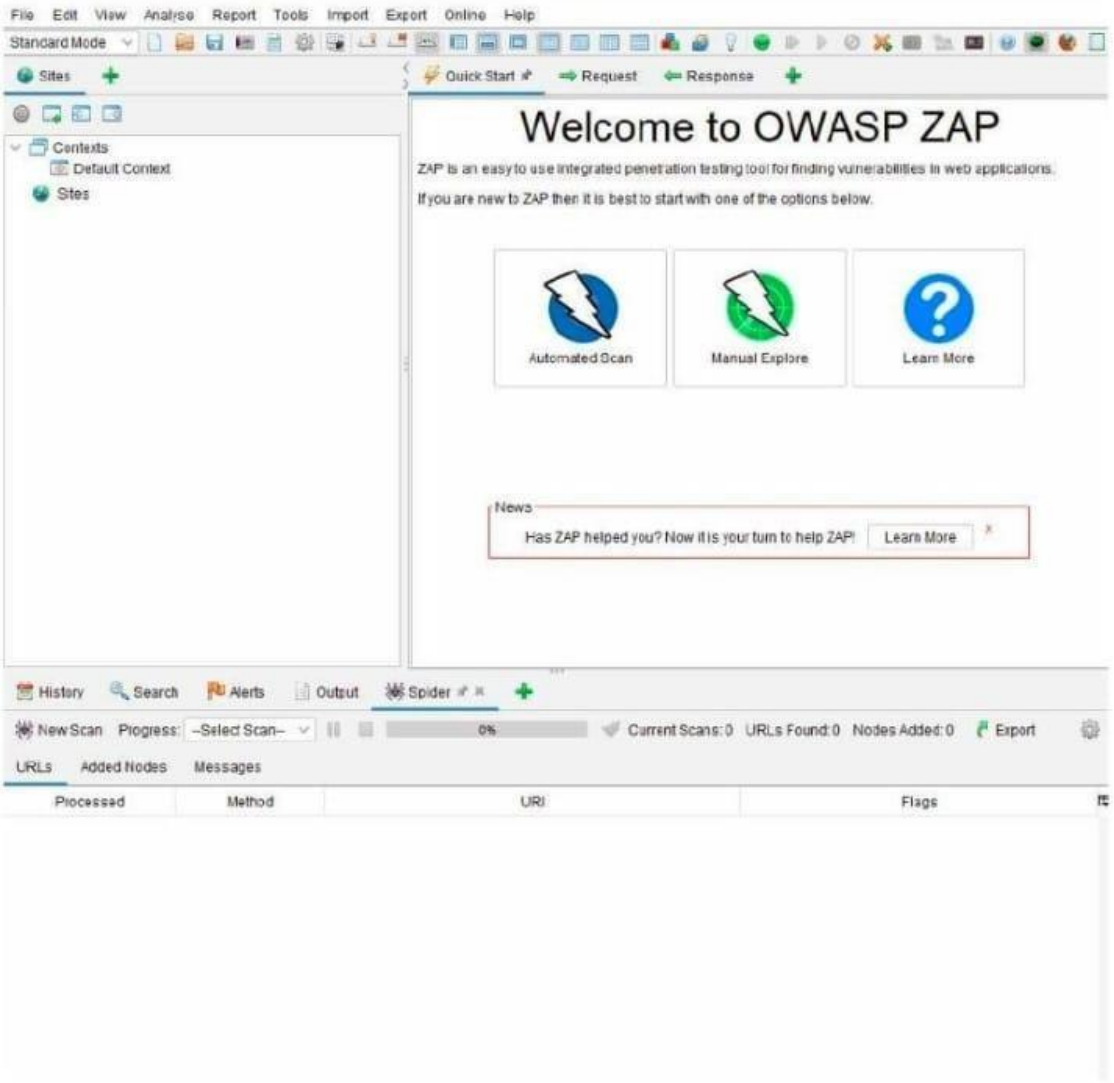
Alternatively, you can persist a session by going to 'File' and choosing 'Persist Session…'. Give your session a name and click on the 'Save' button.



To run an automated scan, you can use the quick start "Automated Scan" option under the "Quick Start" tab. Enter the URL of the site you want to scan in the "URL to attack" field, and then click "Attack!".



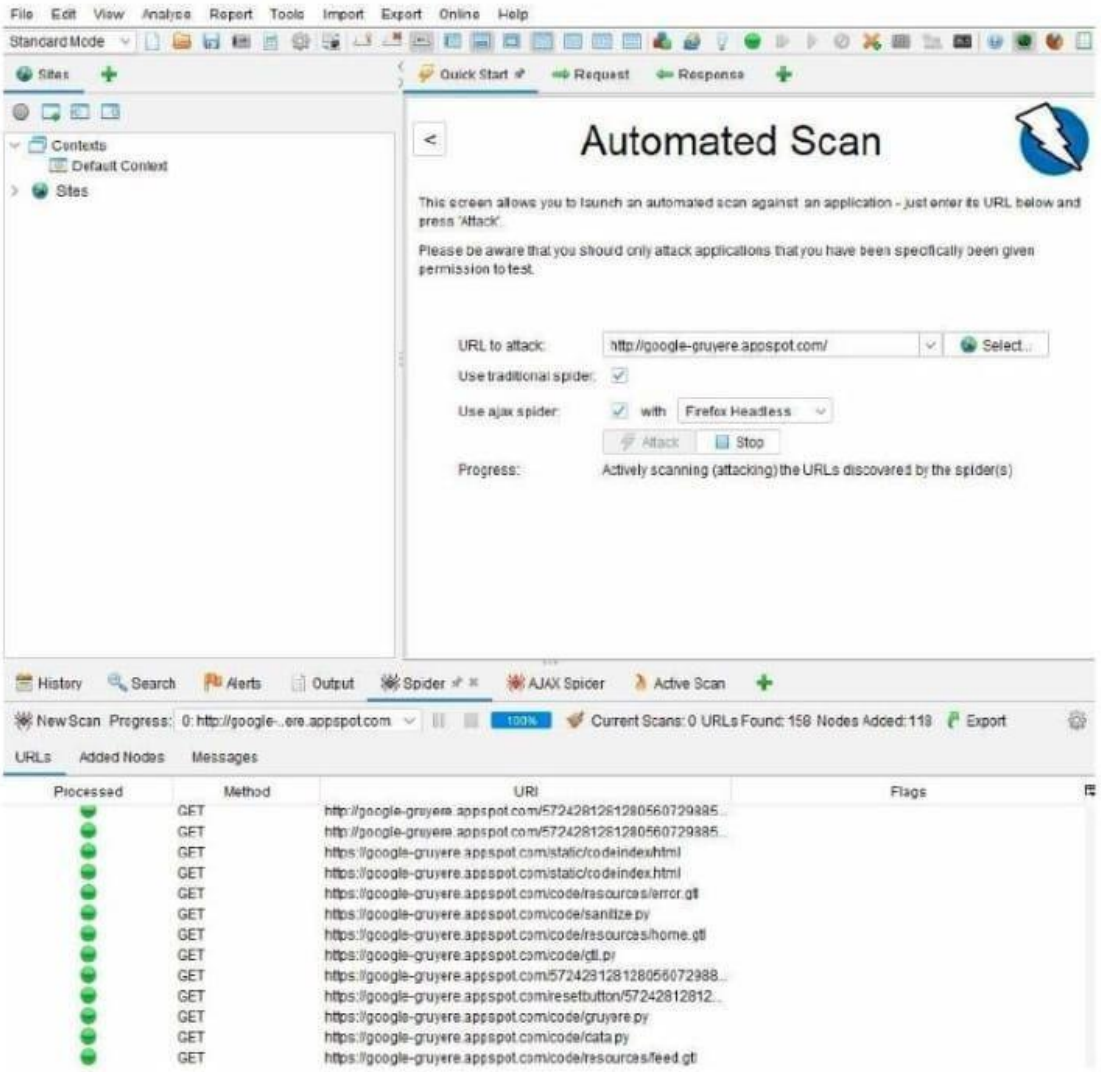
To run an automated scan, you can use the quick start "Automated Scan" option under the "Quick Start" tab. Enter the URL of the site you want to scan in the "URL to attack" field, and then click "Attack!".



4. Interpreting test results

Interpreting test results in OWASP ZAP is vital to understand the scan findings and determine which issues require further investigation. Additionally, it can help to prioritize remediation efforts.

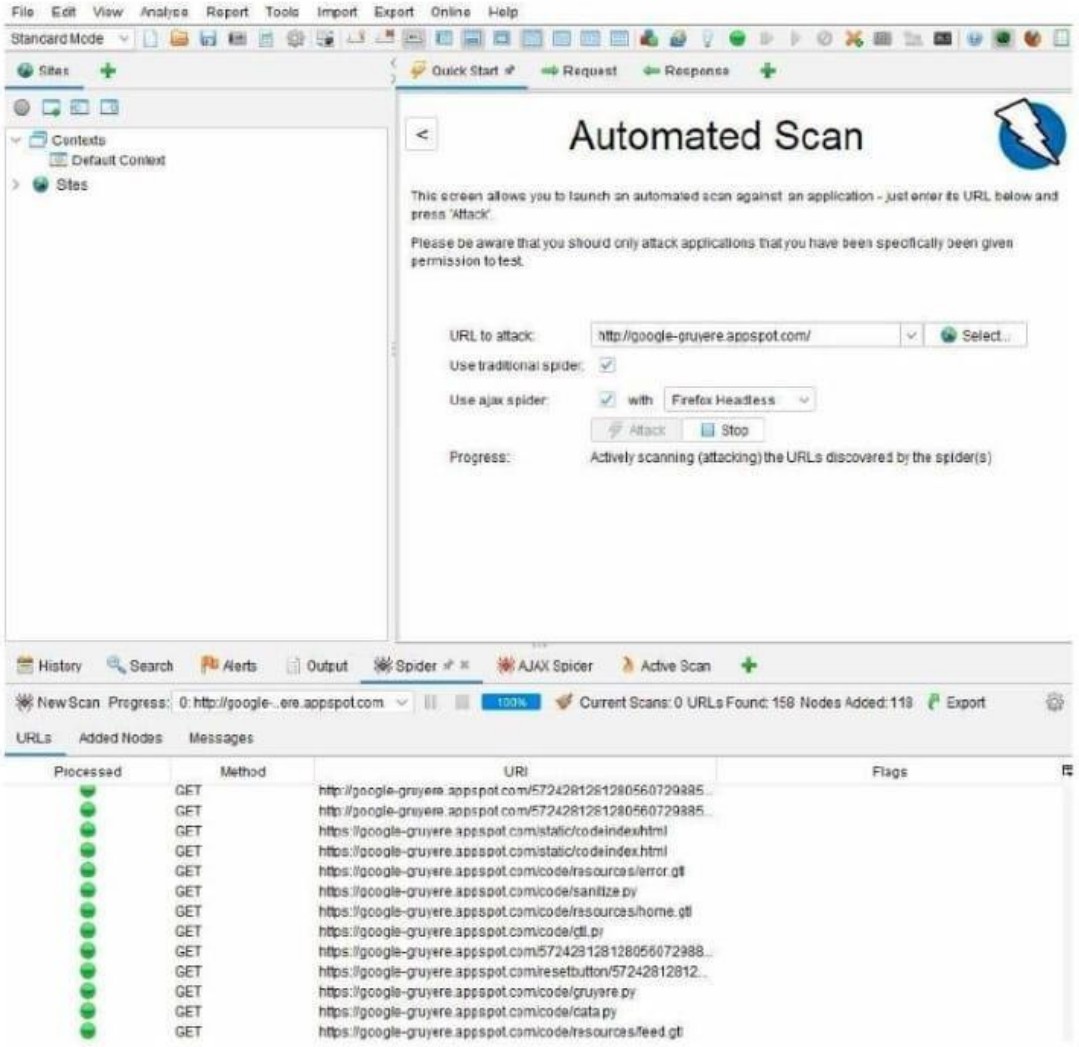
In OWASP ZAP, you can view alerts by clicking on the "Alerts" tab. This tab will show you a list of all the alerts that have been triggered during your testing. The alerts are sorted by risk level, with the highest risk alerts at the top of the list. OWASP ZAP will give details of the discovered vulnerabilities and suggestions on how you can fix them.



5. Viewing alerts and alert details

Viewing alerts and alert details in OWASP ZAP is a way to see what potential security issues have been identified on a website. It can help security and administrators understand what needs to be fixed to improve the app's security.

If you cannot find your 'Alerts' tab, you can access it via the 'View' menu, along with other options available in OWASP ZAP. Once you have your 'Alerts' tab, you can navigate the various vulnerabilities discovered and explore the reports generated by OWASP ZAP.



5. Viewing alerts and alert details

Viewing alerts and alert details in OWASP ZAP is a way to see what potential security issues have been identified on a website. It can help security and administrators understand what needs to be fixed to improve the app's security.

If you cannot find your 'Alerts' tab, you can access it via the 'View' menu, along with other options available in OWASP ZAP. Once you have your 'Alerts' tab, you can navigate the various vulnerabilities discovered and explore the reports generated by OWASP ZAP.

EX.NO:3 Create simple REST API using python for following operation (GET,PUSH,POST,DELETE)

Aim:

To Create simple REST API using python for CRUD operation.

Procedure:

- Install Flask:
- Creating a simple REST API in Python for basic CRUD (Create, Read, Update, Delete) operations typically involves using a web framework.

Code:

1.Install Flask:

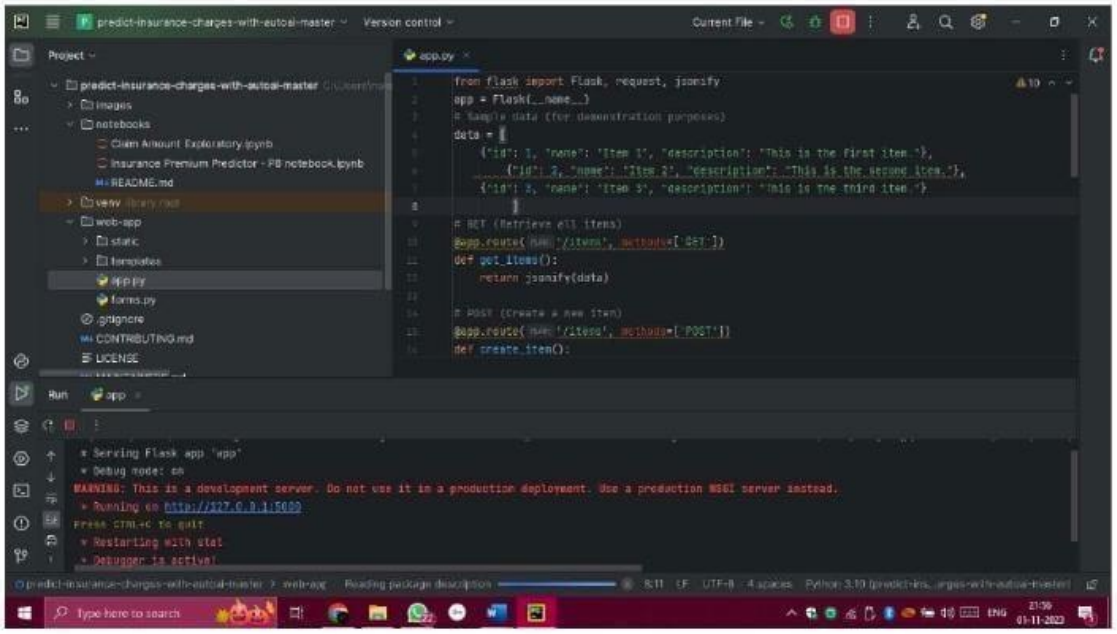
```
pip install flask
```

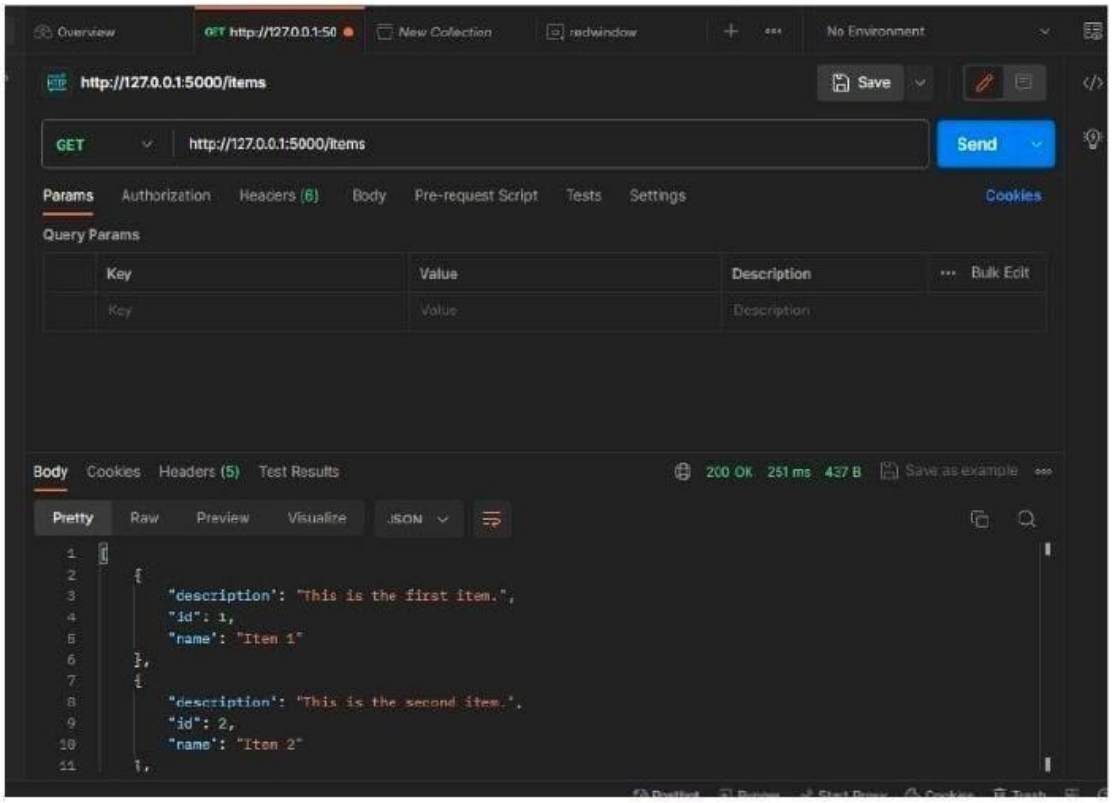
2.Create a new file called app.py:

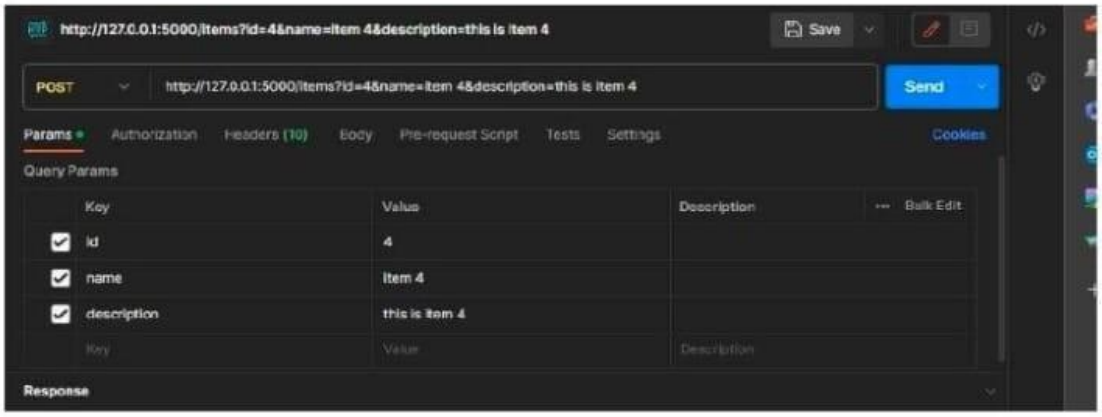
```
from flask import Flask, request, jsonify
app = Flask(__name__)
# Sample data (for demonstration purposes)
data =
  {"id": 1, "name": "Item 1", "description": "This is the first item."},
  {"id": 2, "name": "Item 2", "description": "This is the second item."},
  {"id": 3, "name": "Item 3", "description": "This is the third item."}
    ]
# GET (Retrieve all items)
@app.route('/items', methods=['GET'])
def get_items():
  return jsonify(data)

# POST (Create a new item)
@app.route('/items', methods=['POST'])
def create_item():
  item = request.get_json()
  data.append(item)
```

```
    return jsonify({"message": "Item created successfully"})

# PUT (Update an item)
@app.route('/items/<int:item_id>', methods=['PUT'])
def update_item(item_id):
    if 0 <= item_id < len(data):
        updated_item = request.get_json()
        data[item_id] = updated_item
        return jsonify({"message": "Item updated successfully"})
    else:
        return jsonify({"message": "Item not found"}, 404)

# DELETE (Delete an item)
@app.route('/items/<int:item_id>', methods=['DELETE'])
def delete_item(item_id):
    if 0 <= item_id < len(data):
        del data[item_id]
        return jsonify({"message": "Item deleted successfully"})
    else:
        return jsonify({"message": "Item not found"}, 404)

if __name__ == '__main__':
    app.run(debug=True)
```

Output:


predict-insurance-charges-with-autoai-master
Version control
Current File
Project
app.py
images
notebooks
Claim Amount Exploratory.ipynb
Insurance Premium Predictor - PB notebook.ipynb
README.md
venv
web-app
static
templates
app.py
forms.py
.gitignore
CONTRIBUTING.md
LICENSE
from flask import Flask, request, jsonify
app = Flask(__name__)
# Sample data (for demonstration purposes)
data = [
{'id': 1, 'name': 'Item 1', 'description': 'This is the first item.'},
{'id': 2, 'name': 'Item 2', 'description': 'This is the second item.'},
{'id': 3, 'name': 'Item 3', 'description': 'This is the third item.'}
]
GET (Retrieve all items)
@app.route('/items', methods=['GET'])
def get_items():
return jsonify(data)
POST (Create a new item)
@app.route('/items', methods=['POST'])
def create_item():
Run
app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!


Overview
GET http://127.0.0.1:50
New Collection
redwindow
No Environment
http://127.0.0.1:5000/items
Save
GET
http://127.0.0.1:5000/items
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
251 ms
437 B
Save as example
Pretty
Raw
Preview
Visualize
JSON
[
{
"description": "This is the first item.",
"id": 1,
"name": "Item 1"
},
{
"description": "This is the second item.",
"id": 2,
"name": "Item 2"
},


http://127.0.0.1:5000/items?id=4&name=item 4&description=this is item 4
Save
POST
http://127.0.0.1:5000/items?id=4&name=item 4&description=this is item 4
Send
Params
Authorization
Headers (10)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
id
4
name
item 4
description
this is item 4
Response